

Introduction to Trees

A **tree** is a **non-linear data structure** used to represent data in a **hierarchical form**. It consists of a collection of **nodes** connected by **edges**, where one node acts as the **root**, and every other node is connected through a **parent-child relationship**.

Trees are mainly used to represent data containing a hierarchical relationship between elements, such as:

- File systems
- Organization charts
- Family trees
- XML/HTML documents
- Database indexing

Unlike linear data structures such as arrays or linked lists, trees allow data to be organized in multiple levels.

Definition of Tree

A **tree** is an abstract model of a hierarchical structure that consists of nodes with parent-child relationships.

Important properties:

- A tree is a collection of **nodes**
- One node is designated as the **root**
- Every node except the root has exactly **one parent**
- A node can have **zero or more child nodes**
- Nodes with the same parent are called **siblings**

Basic Terminology of Trees

1. Root Node

The topmost node of the tree is called the **root node**.

Example:

A is the root in the tree.

2. Parent Node

A node that has one or more child nodes is called a **parent node**.

Example:

B is parent of D and E.

3. Child Node

A node directly connected below another node is called a **child node**.

Example:

D and E are children of B.

4. Sibling Node

Nodes that share the same parent are called **siblings**.

Example:

F and G are siblings.

5. Leaf Node

A node that has no children is called a **leaf node**.

Example:

H, I, J, and G are leaf nodes.

6. Subtree

Any node together with its descendants forms a **subtree**.

Example:

Node D with H and I forms a subtree.

7. Level of Node

The number of edges from the root to a node determines its **level**.

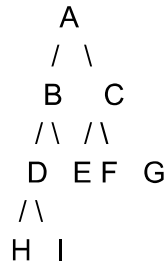
Example:

- Root → Level 0
- Children of root → Level 1
- Grandchildren → Level 2

8. Height of Tree

The maximum level of any node in the tree is called the **height of the tree**.

Example Tree Structure



Here:

- A → Root node
- B, C → Children of A
- D, E → Children of B
- H, I → Children of D
- F, G → Children of C

Characteristics of Trees

- Hierarchical structure
- Dynamic size
- Efficient searching and insertion
- Recursive in nature
- Represents relationships clearly

Applications of Trees

Trees are widely used in computer science for:

- Database indexing using B-Trees
- Expression trees in compilers
- Routing tables in networks
- Decision trees in AI
- File directory systems

Degree

- **Degree of a node:** The degree of a node is the number of children of that node.
- **Degree of a Tree:** The degree of a tree is the maximum degree of nodes in a given tree.

Height and Depth

- **Height of a node:** The height of a node is the max path length from that node to a leaf node.
- **Height of a tree:** The height of a tree is the height of the root.
- **Depth of a tree:** Depth of a tree is the max level of any leaf in the tree.

Types of Trees (On the basis of number of children)

Binary Tree

- A binary Tree is defined as a Tree data structure with at most **2 children**.

Ternary Tree

- A Ternary Tree is a tree data structure in which each node has at most **three child nodes**, usually distinguished as "left", "mid" and "right".

N-Ary Tree

- Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children (duplicate references are not allowed).

Diagram Annotations & Examples

Example Tree Analysis (Image 3)

- **Root of tree (Node 45):** Height of root: 3, Depth of root: 0.
- **Internal Nodes (8, 3, 7):**
 - **Node 8:** Height: 2, Depth: 1, Degree: 2.
 - **Node 3:** Height: 1, Depth: 1, Degree: 1.
 - **Node 7:** Height: 1, Depth: 2.
- **Leaf Nodes (67, 24, 33):**
 - Height: 0.

- Depth of 24, 33: 2.
- Depth of 67: 3.
- **Tree Summary:** "Height of this tree is 3 as length of longest path (45-8-7-67) is 3."

Height and Depth Visualization (Image 4)

- **Node A:** $h=2, d=0$
- **Node B:** $h=1, d=1$
- **Node C:** $h=0, d=1$
- **Node D:** $h=0, d=2$
- **Node E:** $h=0, d=2$